

Deviation Management – Implementation Steps

Do them in order. Steps 9 and 10 are server side, as are 18–22; everything else is client. A row is a deviation only because the user made it one – set at Apply from the baseline diff, or loaded from the stored deviation dates. Never from the configuration file.

Screen, columns and filtering law

Step 1 — WCMModelDisplayProperty

Replace the constant list with this.

```
COMPONENT("MonitoredComponents", 200),
SEVERITY("Severity", 80),
LEVEL("ThresholdLevel", 60),
TYPE("Type", 80),
VALUE("ThresholdValue", 60),
MIN_VALUE("LearningThresholdMinValue", 70),
MAX_VALUE("LearningThresholdMaxValue", 70),
LEARN_TIME("healthAlarmPageColumn8", 130),
WORKCARD_REF("WorkCardRef", 130),
MIN_DEV("LearningThresholdMinDev", 140),
MEAN_MULT_FACTOR("LearningThresholdMeanMultFactor", 170),
DEV_COEF("LearningThresholdDevCoef", 130),
NB_POINTS("NbSuccessivePtForAlarmRaising", 190),
MAX_TIME_WO_ACQ("MaxTimeWithoutAcquisition", 190),
FILTERING_LAW("FilteringLaw", 110),
TE("TeReference", 140);
```

Step 2 — WCMModelDisplayProperty → fromKey

Replace `return null;` at the end with this.

```
throw new IllegalArgumentException("Unknown display property key: " + property);
```

Step 3 — WCTreeLabelProvider → getColumnImage

Delete everything inside the method. Leave only this.

```
return null;
```

Step 4 — WCTreeLabelProvider → getColumnText

In the `WCThresholdModel` branch, after case `NB_POINTS`: block, before `default` :, add this.

```
case FILTERING_LAW:
    return model.getFilteringLawName() != null ? model.getFilteringLawName() : "";
case TE:
    return model.getTeReference() != null ? model.getTeReference() : "";
```

Step 5 — WCTreeLabelProvider → getBackground

Replace the whole method with this.

```

@Override
public Color getBackground(Object element, int column) {
    if (element instanceof WCThresholdModel model) {
        WCModelDisplayProperty property = WCModelDisplayProperty.values()[column];

        if (property == WCModelDisplayProperty.SEVERITY) {
            if (model.getSeverity() == Severity.RED) {
                return CoreResourcesUtils.getColor(CoreResourcesUtils.COLOR_RED);
            } else if (model.getSeverity() == Severity.AMBER) {
                return CoreResourcesUtils.getColor(CoreResourcesUtils.COLOR_AMBER);
            }
            return null;
        }
        if (model.isDeviated()) {
            return CoreResourcesUtils.getColor(CoreResourcesUtils.COLOR_GREEN);
        }
        return model.isOdd() ? ODD_NUMBER_COLOR : EVEN_NUMBER_COLOR;
    }
    return null;
}

```

Step 6 — WCTreeCellModifier → canModify

Find `if (element instanceof WCThresholdModel)`. Change that line and add the guard directly after it.

```

if (element instanceof WCThresholdModel thresholdModel) {
    if (thresholdModel.isDeviated()) {
        return false;
    }
    switch (WCModelDisplayProperty.fromKey(property)) {

```

Step 7 — WCTreeCellModifier → getValue

In the `WCThresholdModel` branch, before `default:`, add this. Also delete the commented-out `CHECKBOX` lines.

```

case FILTERING_LAW:
    return thresholdModel.getFilteringLawName() != null
        ? thresholdModel.getFilteringLawName() : "";

case TE:
    return thresholdModel.getTeReference() != null
        ? thresholdModel.getTeReference() : "";

```

Step 8 — WCTreeCellModifier → modify

Delete the whole case `CHECKBOX:` block. Before `default:`, add this.

```

case FILTERING_LAW:
    if (value != null && !value.toString().trim().isEmpty()) {
        thresholdModel.setFilteringLawName(value.toString());
    } else {
        thresholdModel.setFilteringLawName(null);
    }
    break;
case TE:
    if (value != null && !value.toString().trim().isEmpty()) {
        thresholdModel.setTeReference(value.toString().trim());
    } else {
        thresholdModel.setTeReference(null);
    }
    break;

```

Step 9 — IHealthCommonBusiness (mgs-health-business)

Add this method to the interface. The filtering laws are a `LAZY @OneToMany` on `HealthMonitoringConfiguration`, so they can only be read where the persistence session is open – server side, not in the RCP client.

```

List<String> getFilteringLawNames(String aircraftId) throws EntityNotFoundException;

```

Step 10 — HealthCommonBusiness (implementation)

Add this next to `getLatestConfiguration`. Import `FilteringLaw` from `com.eurocopter.mgs.datamodel.health.monitoring`; everything else is already imported.

```

@Override
public List<String> getFilteringLawNames(String aircraftId) throws EntityNotFoundException {
    List<String> names = new ArrayList<>();
    HealthMonitoringConfiguration configuration = getLatestConfiguration(aircraftId);
    for (FilteringLaw law : configuration.getFilteringLaws()) {
        names.add(law.getName());
    }
    names.sort(String::compareTo);
    return names;
}

```

This class already resolves lazy collections successfully: `getIndicatorModels` walks `getMechanicalComponentConfigurations()` and `getIndicatorConfigurations()` the same way, and that is how the Health tree is built today. Only Strings cross the wire, so nothing lazy reaches the client.

Step 11 — HealthAlarmsPreferenceStore

Add this method anywhere in the class. It supplies the Filtering Law dropdown.

```

public List<String> getFilteringLawNames(Aircraft aircraft) {
    try {
        return healthBusiness.getFilteringLawNames(aircraft.getId());
    } catch (EntityNotFoundException e) {
        LOGGER.warn("Could not load filtering laws", e);
        return new ArrayList<>();
    }
}

```

`healthBusiness` is already a field on this class. No fallback list and no hardcoded law names – an empty dropdown is the honest result if the call genuinely fails.

Do not call `MgsPreferenceStoreManager.getConfiguration(aircraft)` from the client and iterate `getFilteringLaws()`. That returns a detached entity whose `filteringLaws` is an uninstantiated EclipseLink IndirectSet, and iterating it throws "An attempt was made to traverse a relationship using indirection that had a null Session". The same applies to every other collection on that object. Do not change the mapping to `FetchType.EAGER` either – that would load the laws on every configuration fetch across the whole application, including the download monitoring path.

Step 12 — HealthAlarmsPreferenceStore → createAllMechanicalComponents

Find the line starting `List<WCThresholdModel> thresholdModels = getThresholdModels(` and replace it with this.

```

List<WCThresholdModel> thresholdModels = getThresholdModels(healthMechanicalComponent,
    indicatorModels, entry.getValue().getIndicatorConfigurations());

```

Step 13 — HealthAlarmsPreferenceStore → getThresholdModels

Add the third parameter to the signature. Inside the `for (Indicator indicator : sortedSet)` loop, just before `hmcThresholdModels.addAll(...)`, add the block below. Set the default law and the baseline here – **never set deviated from configuration data**.

```

private List<WCThresholdModel> getThresholdModels(
    HealthMechanicalComponent healthMechanicalComponent,
    Map<Indicator, IndicatorModel> indicatorModels,
    Map<Indicator, IndicatorConfiguration> indicatorConfigurations) {

    // inside the loop, before hmcThresholdModels.addAll(...)
    IndicatorConfiguration indicatorConfiguration = indicatorConfigurations.get(indicator);
    String law = indicatorConfiguration != null ? indicatorConfiguration.getFilteringLaw() : null;
    for (WCThresholdModel thresholdModel : indicatorThresholdsMap.values()) {
        thresholdModel.setFilteringLawName(law);
        // Baseline = the applicable configuration. Set once, never updated afterwards.
        thresholdModel.setBaseline(new WCThresholdBaseline(
            thresholdModel.getDefaultValue(), thresholdModel.getMinValue(), law));
    }
}

```

Step 14 — HealthAlarmsPreferenceStore → isValidData

Inside `for (WCThresholdModel currentModel : ...)`, after the switch closes, add this. A row becomes a deviation here, at Apply, because its values differ from the configuration – not on load and not while typing.

```

if (currentModel.isModifiedAgainstBaseline()) {
    currentModel.setDeviated(true);
}

if (currentModel.isDeviated() && (currentModel.getTeReference() == null
    || currentModel.getTeReference().trim().isEmpty())) {
    MessageDialog.openError(parent.getShell(),
        Messages.CORE.getString("healthAlarmPageDataInvalidTitle"),
        MessageFormat.format(Messages.CORE.getString("deviationErrorTeMandatory"),
            currentModel.getIndicator().getName()));
    currentModel.setDeviated(false);
    return false;
}

```

The `setDeviated(false)` before `return false` matters: validation failed, so the row must stay editable for the user to type the TE.

Step 15 — WCThresholdModel

Add this method so the diff lives in one place.

```

public boolean isModifiedAgainstBaseline() {
    if (baseline == null) {
        return false;
    }
    return !Objects.equals(defaultValue, baseline.getDefaultValue())
        || !Objects.equals(minValue, baseline.getMinValue())
        || !Objects.equals(filteringLawName, baseline.getFilteringLawName());
}

```

Import `java.util.Objects`. Match the getter names to your own `WCThresholdBaseline`. Nothing may ever write to the baseline after `getThresholdModels` sets it – not the cell modifier, not Apply.

Step 16 — WCModelTreeViewer

Add the field at the top of the class, replace the `FILTERING_LAW` editor line in `createCellEditors`, and add two lines to `setSelectedAircraft`.

```

private ComboBoxViewerCellEditor filteringLawEditor;

// in createCellEditors, replacing:
// editors[WCModelDisplayProperty.FILTERING_LAW.ordinal()] = getCellEditor(tree, true);
filteringLawEditor = new ComboBoxViewerCellEditor(tree, SWT.READ_ONLY);
filteringLawEditor.setLabelProvider(new LabelProvider() {
    @Override
    public String getText(Object element) {
        return element == null ? "" : element.toString();
    }
});
filteringLawEditor.addListener(createWCColumnListener());
filteringLawEditor.setContentProvider(new ArrayContentProvider());
editors[WCModelDisplayProperty.FILTERING_LAW.ordinal()] = filteringLawEditor;

// in setSelectedAircraft, after this.selectedAircraft = selectedAircraft;
if (filteringLawEditor != null && selectedAircraft != null) {
    filteringLawEditor.setInput(healthAlarmsPreferenceStore.getFilteringLawNames(selectedAircraft));
}

```

Step 17 — messages.properties – and every language file

Add these keys.

```

FilteringLaw=Filtering Law
TeReference=TE *
deviationErrorTeMandatory=Error : deviation can't be set on indicator {0}. Reason : TE field is mandatory
deviationSuccess=Deviation successfully set on indicator {0}
deviationAlreadySet=A deviation is already set on this indicator.

```

Rebuild the business layer and redeploy once for Steps 9 and 10, then run the app. Columns are wide, no tick box, the Filtering Law dropdown lists the real laws from the configuration, and rows the configuration links to a law show it. Edit a value, type a TE and press Apply: the row turns green and locks. Press Apply with an empty TE and it is rejected and stays editable.

Persistence – entity, DTOs and delegate

Step 18 — ThresholdConfiguration (mgs-jpa-model/src/main/java)

After the workcard field, add these.

```
@Column(name = "TE_REFERENCE")
private String teReference;

@Temporal(TemporalType.TIMESTAMP)
@Column(name = "DEVIATION_START_DATE")
private Date deviationStartDate;

@Temporal(TemporalType.TIMESTAMP)
@Column(name = "DEVIATION_END_DATE")
private Date deviationEndDate;

public String getTeReference() { return teReference; }
public void setTeReference(String teReference) { this.teReference = teReference; }
public Date getDeviationStartDate() { return deviationStartDate; }
public void setDeviationStartDate(Date d) { this.deviationStartDate = d; }
public Date getDeviationEndDate() { return deviationEndDate; }
public void setDeviationEndDate(Date d) { this.deviationEndDate = d; }

public boolean isDeviationActive() {
    return deviationStartDate != null && deviationEndDate == null;
}
```

Step 19 — DDL / migration script

Add the three columns.

```
ALTER TABLE THRESHOLD_CONFIGURATION ADD TE_REFERENCE VARCHAR(255);
ALTER TABLE THRESHOLD_CONFIGURATION ADD DEVIATION_START_DATE TIMESTAMP;
ALTER TABLE THRESHOLD_CONFIGURATION ADD DEVIATION_END_DATE TIMESTAMP;
```

Step 20 — ThresholdConfigurationDto (mgs-services-interfaces)

After the workcard field, add these.

```
private String teReference;
private Date deviationStartDate;
private Date deviationEndDate;

public final String getTeReference() { return teReference; }
public final void setTeReference(String teReference) { this.teReference = teReference; }
public final Date getDeviationStartDate() { return deviationStartDate; }
public final void setDeviationStartDate(Date d) { this.deviationStartDate = d; }
public final Date getDeviationEndDate() { return deviationEndDate; }
public final void setDeviationEndDate(Date d) { this.deviationEndDate = d; }
```

Step 21 — IndicatorConfigurationDto (mgs-services-interfaces)

After the thresholdConfigurationDtos field, add this.

```
private String filteringLaw;

public final String getFilteringLaw() { return filteringLaw; }
public final void setFilteringLaw(String filteringLaw) { this.filteringLaw = filteringLaw; }
```

Step 22 — HealthDelegate → updateHealthConfiguration (mgs-delegate)

After IndicatorConfiguration indicatorConfiguration = indicatorConfigurationsMap.get(...); add the first line. After thresholdConfiguration.setWorkcard(...); add the other three.

```
indicatorConfiguration.setFilteringLaw(indicatorConfigurationDto.getFilteringLaw());

thresholdConfiguration.setTeReference(thresholdConfigurationDto.getTeReference());
thresholdConfiguration.setDeviationStartDate(thresholdConfigurationDto.getDeviationStartDate());
thresholdConfiguration.setDeviationEndDate(thresholdConfigurationDto.getDeviationEndDate());
```

Step 23 — WCThresholdModel

Add two fields with getters and setters.

```
private Date deviationStartDate;  
private Date deviationEndDate;
```

Step 24 — HealthAlarmsPreferenceStore → innerSave

After `IndicatorConfigurationDto indicatorConfigurationDto = new IndicatorConfigurationDto(...)`; add the first line. After `thresholdConfigurationDto.setWorkcard(...)`; add the rest.

```
indicatorConfigurationDto.setFilteringLaw(wcThresholdModel.getFilteringLawName());  
  
thresholdConfigurationDto.setTeReference(wcThresholdModel.getTeReference());  
thresholdConfigurationDto.setDeviationStartDate(  
    wcThresholdModel.getDeviationStartDate() != null  
        ? wcThresholdModel.getDeviationStartDate()  
        : (wcThresholdModel.isDeviated() ? new Date() : null));  
thresholdConfigurationDto.setDeviationEndDate(wcThresholdModel.getDeviationEndDate());
```

Step 25 — HealthAlarmsPreferenceStore → getThresholdModels

In the same block as Step 11, add the load-back. This is the **only** place `deviated` is set from stored data, and it reads the persisted dates – never the configuration file.

```
thresholdModel.setTeReference(thresholdConfiguration.getTeReference());  
thresholdModel.setDeviationStartDate(thresholdConfiguration.getDeviationStartDate());  
thresholdModel.setDeviationEndDate(thresholdConfiguration.getDeviationEndDate());  
thresholdModel.setDeviated(thresholdConfiguration.isDeviationActive());
```

Rebuild in this order: `mgs-jpa-model`, `mgs-services-interfaces`, `mgs-delegate`, then the client. Redeploy the EAR. Batch this with Steps 9 and 10 so the server is redeployed only once.

Three tabs – client only

Step 26 — HealthAlarmsComposite – imports

Add these three imports.

```
import org.eclipse.swt.widgets.TabFolder;  
import org.eclipse.swt.widgets.TabItem;  
import com.eurocopter.mgs.core.presentation.i18n.Messages;
```

Step 27 — HealthAlarmsComposite → constructor

Find `createTreeViewer(this);` and replace that single line with this block. The A/C combo and the TE message stay above, untouched.

```
TabFolder tabFolder = new TabFolder(this, SWT.NONE);  
tabFolder.setLayoutData(new GridData(SWT.FILL, SWT.FILL, true, true));  
  
// Tab 1 - Configuration: holds the existing tree  
TabItem configurationTab = new TabItem(tabFolder, SWT.NONE);  
configurationTab.setText(Messages.CORE.getString("deviationTabConfiguration"));  
Composite configurationComposite = new Composite(tabFolder, SWT.NONE);  
GridLayout tabLayout = new GridLayout();  
tabLayout.marginHeight = 0;  
tabLayout.marginWidth = 0;  
configurationComposite.setLayout(tabLayout);  
configurationTab.setControl(configurationComposite);  
createTreeViewer(configurationComposite);  
  
// Tab 2 - Deviation list (empty for now)  
TabItem deviationListTab = new TabItem(tabFolder, SWT.NONE);  
deviationListTab.setText(Messages.CORE.getString("deviationTabList"));  
deviationListTab.setControl(new Composite(tabFolder, SWT.NONE));  
  
// Tab 3 - Deviation history (empty for now)  
TabItem deviationHistoryTab = new TabItem(tabFolder, SWT.NONE);  
deviationHistoryTab.setText(Messages.CORE.getString("deviationTabHistory"));  
deviationHistoryTab.setControl(new Composite(tabFolder, SWT.NONE));
```

Step 28 — messages.properties – and every language file

Add these three keys.

```
deviationTabConfiguration=Configuration  
deviationTabList=Deviation list  
deviationTabHistory=Deviation history
```

createTreeView already takes a Composite parameter, so it needs no change. Tabs 2 and 3 are empty placeholders – the tables inside them are a separate ticket, as is the Manage Filtering Laws dialog.

Known problem, deferred

HealthAlarmsPreferenceStore.innerSave and getComponentThresholds both skip rows where isEnabled() is false, and WCThresholdModel defaults it to FALSE. With the tick box gone the user cannot enable a row, so a deviation on an unticked row will not save. Mark both call sites with a TODO.